4 - Estrategias de evaluación 🧩

Tabla de contenidos

1 Introducción 1
2 Evaluación estricta 1
3 Evaluación inmediata 2
4 Evaluación no estricta 3
4.1 Expresión `and` 3
4.2 Expresión `or` 4
4.3 Expresiones condicionales 5
5 Evaluación perezosa 5

1 Introducción

En este breve capítulo vamos a explorar las diferentes estrategias de evaluación que utiliza Python. En programación, una estrategia de evaluación es el conjunto de reglas que define cómo y cuándo se calculan las expresiones.

Analizaremos la **evaluación inmediata** (*eager*) y la **evaluación perezosa** (*lazy*), así como la diferencia entre **evaluación estricta** y **no estricta**.

2 Evaluación estricta

Supongamos una función a la que le pasamos dos números `a` y `b` y nos devuelve una lista con todos los enteros entre `a` y `b`.

```
def intervalo(a, b):
    return list(range(a, b + 1))

intervalo(5, 12)
```

```
[5, 6, 7, 8, 9, 10, 11, 12]
```

Para cumplir su objetivo, esta función solo necesita los valores `a` y `b`. Pero, ¿qué pasaría si por error le agregamos otro parámetro y después la llamamos sin darle ningún valor para ese parámetro?

```
def intervalo(a, b, c):
    return list(range(a, b + 1))
```

```
intervalo(5, 12)
```

```
    intervalo(5, 12)
    ~~~~~~~~~^^^^^^^
TypeError: intervalo() missing 1 required positional argument: 'c'
```

Python devuelve un error indicando que la función `intervalo` no puede ejecutarse porque falta un valor para el parámetro `c`. Aunque sepamos que dicho parámetro no se utiliza dentro de la función, Python igualmente exige que se le pase un valor.

Esta exigencia se debe a que las funciones de Python se evalúan bajo las reglas de la **evaluación estricta**. Bajo este enfoque, una función no puede producir un resultado si alguno de sus parámetros no está definido. Por eso, aun cuando el valor de `c` nunca se use en el cuerpo de la función, la ausencia de un valor para él provoca un error.

Para que la función se ejecute correctamente, basta con pasarle cualquier valor:

```
intervalo(5, 12, None)
```

```
[5, 6, 7, 8, 9, 10, 11, 12]
```

En Python, **todas** las llamadas a funciones siguen la estrategia de evaluación estricta, y esto no es algo que podamos modificar: forma parte del diseño del lenguaje. De todas formas, como veremos más adelante, existen ciertas construcciones del propio lenguaje que aplican una estrategia de evaluación no estricta.

3 Evaluación inmediata

Otro aspecto interesante al trabajar con funciones es el **momento** en que se evalúan los argumentos que se pasan en las llamadas.

Tomemos de nuevo nuestra función original:

```
def intervalo(a, b):
    return list(range(a, b + 1))
```

y la siguiente llamada:

```
intervalo(1 + 3, 5 + 12)
```

La pregunta es: ¿qué pasa primero? ¿Se ejecuta la función `intervalo` y recién ahí se resuelven las expresiones `1 + 3` y `5 + 12`, o esas expresiones se calculan antes y luego se pasan sus resultados a la función?

Para aclarar esta duda, en lugar de usar una suma directa podemos probar con una función que sume pero que, además, imprima los argumentos que recibe. Del mismo modo, podemos modificar la función `intervalo` para que muestre un mensaje cuando sea llamada y así ver con más claridad el orden en que ocurren las cosas.

```python
def suma(x, y):
    print(f"suma(x={x}, y={y})")
    return x + y


def intervalo(a, b):
    print(f"intervalo(a={a}, b={b})")
    return list(range(a, b + 1))

intervalo(suma(1, 3), suma(5, 12))
```

```
suma(x=1, y=3)
suma(x=5, y=12)
intervalo(a=4, b=17)
[4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17]
```

Por el orden en que aparecen los mensajes impresos, vemos que Python primero ejecutó la llamada `suma(1, 3)`, luego `suma(5, 12)` y recién después invocó a la función `intervalo`, ya recibiendo los valores resultantes de las llamadas a `suma`.

En otras palabras, antes de llamar a `intervalo`, Python evaluó las expresiones que se pasaron como argumentos.

A esta estrategia de evaluación, donde los argumentos se resuelven antes de ejecutar la función, se la conoce como **evaluación inmediata**, o por su nombre en inglés, *eager evaluation*.

4 Evaluación no estricta

En contraste con la evaluación estricta, la evaluación no estricta es una estrategia en la que se puede determinar el resultado de una expresión sin necesidad de evaluar todos sus argumentos u operandos.

En Python, solo tres tipos de expresiones siguen esta estrategia: `and`, `or` y las expresiones condicionales.

En los siguientes ejemplos, vamos a utilizar una función que imprime el valor ingresado y lo devuelve:

```python
def f(x):
    print(f"el valor es {x}")
    return x
```

4.1 Expresión `and`

Una expresión `and` devuelve `True` únicamente cuando todos sus argumentos son verdaderos. Por eso, en el siguiente ejemplo, se imprime `el valor es True` antes de obtener el resultado de la operación `and`, ya que fue necesario evaluar la función `f`.

```
True and f(True)
```

```
el valor es True
True
```

En cambio, cuando el primero de sus argumentos es `False`, se puede anticipar que el resultado de la operación `and` será `False`, sin necesidad de evaluar el segundo argumento. Por este motivo, la función `f` no se ejecuta en ninguno de los siguientes casos, al punto de que podemos pasarle argumentos absurdos:

```
False and f(True)
```

```
False
```

```
False and f(1 / 0)
```

```
False
```

En Python, este mecanismo también se conoce como cortocircuito. Al detectar que el resultado de la operación ya está decidido con el primer argumento, el intérprete "corta camino" y decide no evaluar al resto.

4.2 Expresión `or`

El mecanismo de cortocircuito también funciona para el operador `or`:

```
# El primer argumento es True, el resultado es True
True or f(True)
```

```
True
```

```
# El primer arugmento es False, hay que evaluar el segundo para determinar el
resultado
False or f(True)
```

```
el valor es True
True
```

Las mismas reglas de evaluación se mantienen en expresiones más complejas. En el siguiente ejemplo, es necesario ejecutar `f(True)` para poder resolver el primer `or`, pero no hace falta evaluar `1 + [2, 3]`, que provocaría un error, porque el valor `True` ya alcanza para determinar el resultado de toda la operación.

```
False or f(True) or (1 + [2, 3])
```

```
el valor es True
True
```

4.3 Expresiones condicionales

Finalmente, esta estrategia de evaluación no estricta (o cortocircuito) también se aplica en las expresiones condicionales.

En el siguiente ejemplo, la expresión devuelve `10` porque la condición es verdadera, sin necesidad de ejecutar `f(20)`.

```
10 if True else f(20)
```

```
10
```

En cambio, esta segunda expresión sí requiere que se ejecute la llamada en la parte `else` de la expresión.

```
10 if False else f(20)
```

```
el valor es 20
20
```

5 Evaluación perezosa

Para terminar, veamos una última estrategia de evaluación: la **evaluación perezosa** (en inglés, *lazy evaluation*).

Partimos de una función que recibe un valor, imprime un mensaje con él y lo devuelve:

```
def identidad(x):
    print("Devuelvo", x)
    return x
```

Si usamos `map` para aplicar la función `identidad` a los números 0, 1, 2 y 3:

```
map_obj = map(identidad, range(4))
map_obj
```

```
<map at 0x7fedb41f3cd0>
```

vemos que se crea un objeto `map`, pero no aparece ningún mensaje de la función `identidad`. Esto pasa porque `map` no ejecuta la función sobre los elementos hasta que realmente hace falta.

Por ejemplo, si convertimos el objeto `map` en una lista, recién ahí se produce la evaluación:

```
lista = list(map_obj)
```

```
Devuelvo 0
Devuelvo 1
Devuelvo 2
Devuelvo 3
```

```
lista
```

```
[0, 1, 2, 3]
```

A esta estrategia, en la que se retrasa la evaluación de las expresiones hasta el último momento posible, se la llama **evaluación perezosa** o *lazy evaluation*.

De manera similar, `filter` también utiliza una estrategia de evaluación perezosa. Recién cuando queremos materializar los elementos se aplica la función de filtro.

```
def es_multiplo_5(x):
    print(f"x={x}")
    return x % 5  ==0

filter_obj = filter(es_multiplo_5, [12, 10, 8, 5, 125, 55, 11, 9])
```

```
for x in filter_obj:
    print(f"El numero {x} es múltiplo de 5")
```

```
x=12
x=10
El numero 10 es múltiplo de 5
x=8
x=5
El numero 5 es múltiplo de 5
x=125
El numero 125 es múltiplo de 5
x=55
El numero 55 es múltiplo de 5
x=11
x=9
```

Como se puede ver, la función `es_multiplo_5` se ejecutó para cada valor de la lista numérica, pero `filter` únicamente devolvió aquellos que efectivamente son múltiplos de 5.

En el próximo y último capítulo vamos a explorar los **generadores**, un tipo especial de iterador que implementa una estrategia de evaluación perezosa y nos permite recorrer secuencias incluso potencialmente infinitas.